

Herança e Polimorfismo

Eduardo Piveta

OO

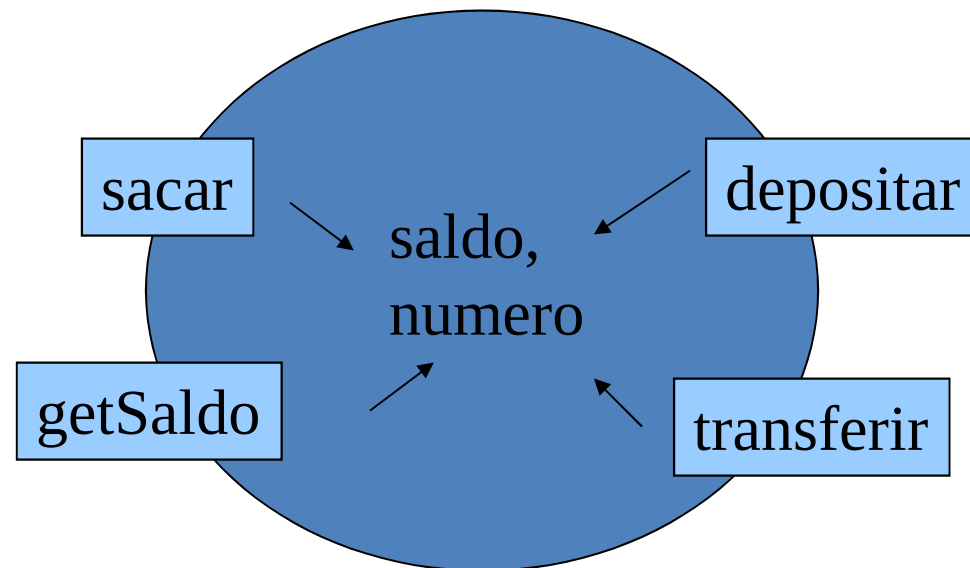
- Divisão de um sistema em módulos (classes) que representam entidades do mundo real com suas características (atributos) e comportamentos (métodos)

Encapsulamento

- Abstração de um conjunto de atributos e métodos em uma classe. (manipulável somente através de sua interface)
- Encapsulamento de código em uma única idéia que pode ser entendível sem detalhes de implementação
 - Subrotinas

Exemplo de Encapsulamento

Classe Conta

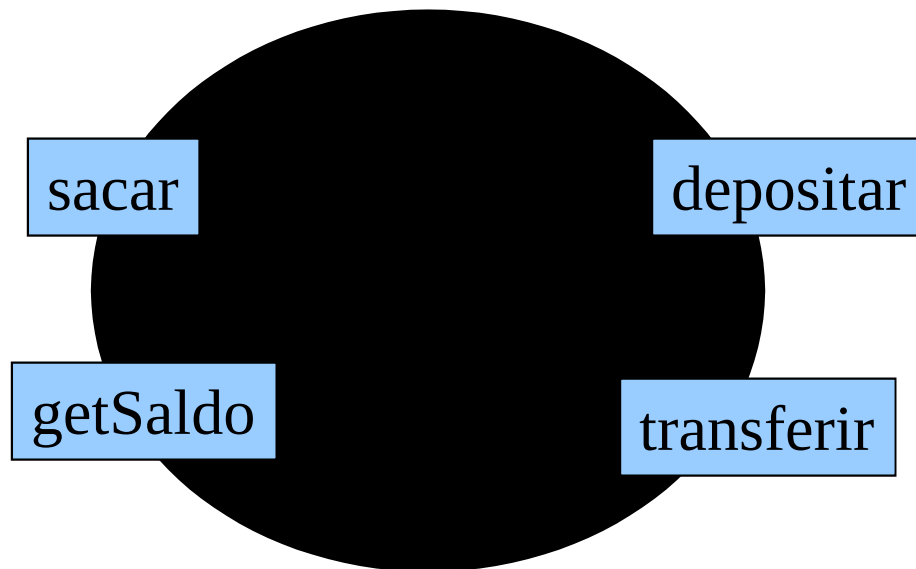


Ocultamento de Informações

- Utilização de encapsulamento para restringir a visibilidade externa dos detalhes de implementação

Exemplo de Ocultamento

CONTA



Retenção de Estado

- Um objeto armazena seu estado por tempo indeterminado
 - Estado: definido através do conjunto dos valores dos atributos de um objeto
- Exemplo:
 - `c1.setSaldo(100);`
 - `System.out.println(c1.getSaldo()); // 100`
 - `c1.sacar(10);`
 - `System.out.println(c1.getSaldo()); // 90`

Identidade de Objeto

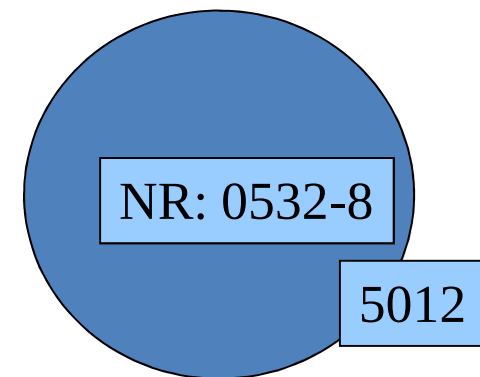
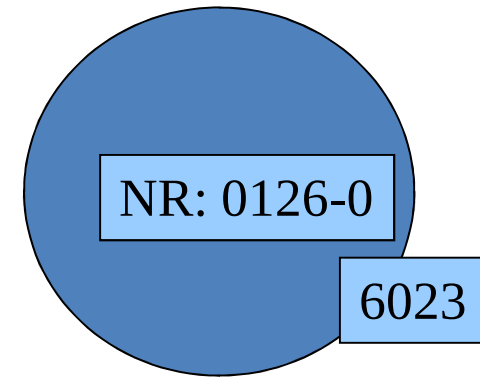
- Propriedade pela qual um objeto é distinguido dos demais
- Observações:
 1. O mesmo identificador permanece igual durante a vida de um objeto
 2. Dois objetos nunca podem ter o mesmo identificador

Exemplo de Identidade

Conta c1, c2;
c1 = new Conta("0126-0");
c2 = new Conta("0532-8");

c1: 6023

c2: 5012



Exemplo de Identidade

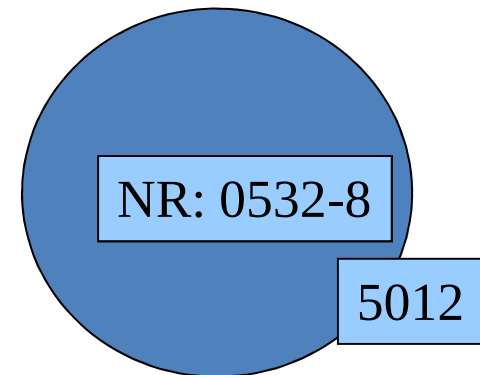
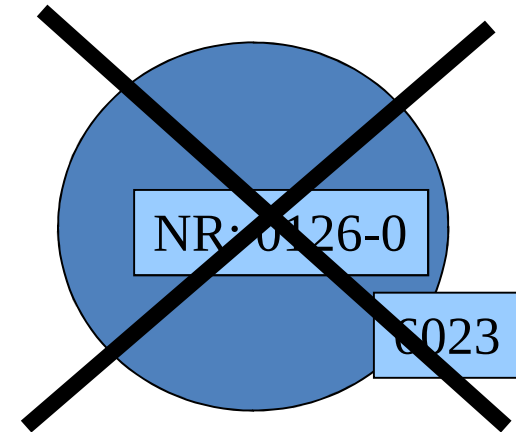
```
c1 = c2;
```

```
c1: 5012
```

```
c2: 5012
```

```
c1.getNr(); // 0532-8;  
c2.getNr(); // 0532-8
```

Existem duas variáveis de referências, mas um único objeto com identidade = 5012

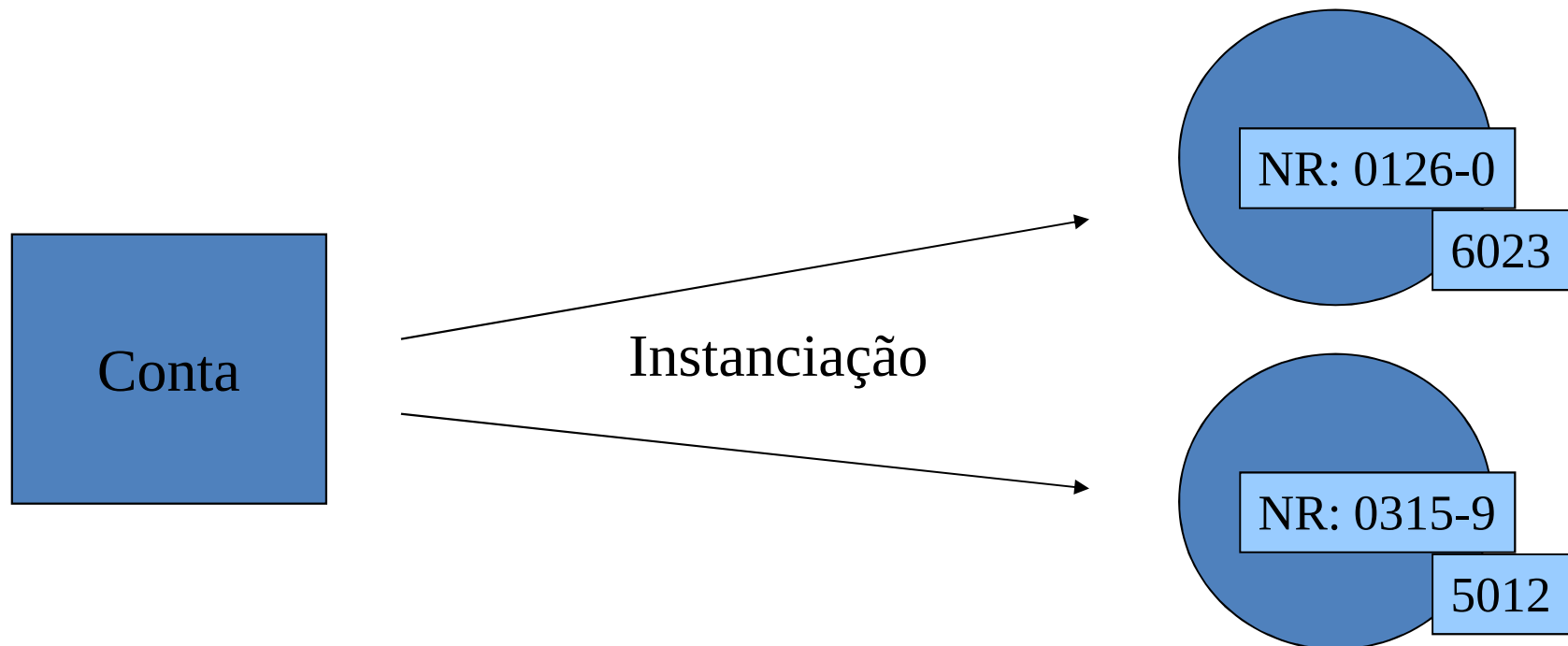


Mensagens

- Mecanismo utilizado para solicitar que um objeto realize operações
- Estruturado:
 - `getSaldo(c1);`
- Orientado a objetos:
 - `c1.getSaldo();`

Classe

- Mecanismo utilizado para a geração de objetos. Descreve os atributos e métodos dos objetos através dela gerados



Herança

- Possibilita a adição e alteração de atributos e comportamentos de uma classe de maneira aditiva.
- Em Java, a herança é especificada através da palavra reservada **extends**
- Sintaxe:

```
public class SubClasse extends SuperClasse{ }
```

Exemplo de Herança

```
class Conta{  
    double saldo;  
    void sacar(double valor){  
        saldo -= valor;  
    }  
}
```

```
class ContaCorrente extends Conta{  
    void sacar(double valor){  
        super(valor);  
        saldo -= valor * 0.0038;  
    }  
}
```

Herança

- Através de herança, pode-se:
 - Usar atributos e métodos não privados da super-classe.
 - Definir e implementar novos atributos e métodos em uma sub-classe.
 - Realizar sobrescritas (**overriding**):
 - Estender o comportamento de métodos da super-classe.
 - Sobrescrever o comportamento de métodos da super-classe.

Uso de atributos e métodos da super-classe

- Os atributos e métodos da super-classe podem ser acessados normalmente nas sub-classes, desde que não sejam privados.

```
public class SuperClasse {  
    public int atributo;  
    public void metodo(){}  
}
```

```
public class SubClasse  
    extends SuperClasse{}
```

Em algum método (em um main, por exemplo):

```
SubClasse s = new SubClasse();  
s.atributo = 10;  
s.metodo();
```


Definição e implementação novos atributos e métodos

- As sub-classes podem definir novos atributos e definir e implementar novos métodos

```
public class SubClasse extends SuperClasse{  
    public int novoAtributo;  
    public void novoMetodo(){};  
}
```

Em algum método:

```
SubClasse s = new SubClasse();  
s.atributo = 10;  
s.metodo();  
s.novoAtributo = 20;  
s.novoMetodo();
```

Exercício

1. Crie uma classe **Veiculo** com os seguintes atributos e métodos.

– Atributos:

- **marca**: marca do veículo.
- **modelo**: modelo do veículo.

– Métodos:

- **ligar()**: mostra a mensagem "Veiculo ligado".
- **desligar()**: mostra a mesma mensagem anterior, trocando "ligado" por "desligado".

Exercício (continuação)

- a) Crie duas subclasses: **Moto** e **Carro**. Para cada uma delas, exemplifique o uso dos atributos **marca**, **modelo** e dos métodos **ligar()** e **desligar()**.
- b) Para a classe **Moto** defina o atributo **numeroRodas**. Para a classe **Carro** defina o atributo **tamanhoPortaMalas**. Exemplifique o uso destes atributos.

Sobrescrita - Extensão

- Pode-se estender os métodos da sub-classe...

```
public class SuperClasse {  
    public int atributo;  
    public void metodo(){  
        atributo++;  
    }  
}
```

```
public class SubClasse  
    extends SuperClasse{  
    public void metodo(){  
        super.metodo();  
        atributo += 10;  
    }  
}
```

Em algum método:

```
SuperClasse s = new SuperClasse();  
s.atributo = 10;  
s.metodo(); // 11
```

Em algum método:

```
SubClasse s = new SubClasse();  
s.atributo = 10;  
s.metodo(); // 21
```

Sobrescrita - Substituição

- ... ou sobrescrevê-los.

```
public class SuperClasse {  
    public int atributo;  
    public void metodo(){  
        atributo++;  
    }  
}
```

```
public class SubClasse  
    extends SuperClasse{  
    public void metodo(){  
        atributo += 10;  
    }  
}
```

Em algum método:
SuperClasse s = new SuperClasse();
s.atributo = 10;
s.metodo(); // 11

Em algum método:
SubClasse s = new SubClasse();
s.atributo = 10;
s.metodo(); // 20

Exercício

- c) Substitua o método `toString()` da super-classe `Object` por um que mostre a `marca` e o `modelo` de um `Veiculo`. Exemplifique o uso deste método.
- d) Substitua os métodos `ligar()` e `desligar()` nas sub-classes `Moto` e `Carro` de forma que eles mostrem, respectivamente `"Moto ligada"` ou `"Moto desligada"` e `"Carro ligado"` ou `"Carro desligado"`.

Exercício

- e) Nas classes **Moto** e **Carro**, estenda o método **toString()** de forma a mostrar também informações sobre o número de rodas e o tamanho do porta malas.
- f) Crie uma classe **Triciclo**, sub-classe de **Moto**. Faça com que o construtor defina como padrão o número de rodas como sendo 3. Estenda o método **ligar** de forma a mostrar:
"Moto ligada"
"Este eh um triciclo".

Polimorfismo

- A: Habilidade pela qual uma única operação pode ser definida em mais de uma classe e assumir diferentes implementações em cada classe.
- Exemplos:
 - métodos toString(), ligar() e desligar().

Polimorfismo

- B: Habilidade de um atributo ou variável poder apontar para objetos de diferentes classes ao longo do tempo .
- Exemplo:
 - `Moto m = new Moto();`
 - `Veiculo v = m;`
 - `Carro c = new Carro();`
 - `v = c;`

Polimorfismo

- **Princípio de Liskov** – um objeto de uma subclasse poder ser sempre atribuído para uma variável de uma de suas superclasses.
 - O contrário nem sempre é verdade, e precisa de uma conversão explícita. Exemplo:
 - `Veiculo v = new Veiculo();`
 - `Moto m = (Moto) v; // ClassCastException.`
- **Ligação Tardia** – mecanismo pelo qual as chamadas a um método só são determinadas em tempo de execução.

Exercício

- g) Crie uma classe chamada **Oficina**, com o seguinte método:
 - `public void testar(Veiculo v) { v.ligar(); v.desligar(); }`.

Exemplifique o uso deste método quando este receber uma **Moto**, um **Carro**, um **Veiculo** e um **Triciclo**.

Polimorfismo

- **Sobrecarga (*Overloading*)** - possibilidade de um método conter diferentes comportamentos de acordo com a sua assinatura

Exercício

- h) Na classe **Oficina**, crie os seguintes métodos:
 - `desmontar(Moto m) { System.out.println("Moto " + m.getMarca() + " desmontada");}`
 - `desmontar(Carro c) { System.out.println("Desmontando carro " + c.getMarca() + " : " + c.getModelo());}`

Exemplifique o uso de tais métodos.